The Big Picture

1)FLOW

Basic pipeline-
https://drive.google.com/file/d/1gVrYOCOqCiu6lPgUyUiajgVvUoI5rQ50/view?usp=sharing

Detailed pipeline-
https://drive.google.com/file/d/1xk1gczJM_7piNf3--KHW_PYUh0vxHzsC/view?usp=sharing

2)CLUSTERS

Cluster_0_overview.png - cluster_0_overview.png

Cluster_1_site_constraints.png- cluster_1_site_constraints.png

Cluster_2a_footprint.png - cluster_2a_footprint.png

Cluster_2b_floorplan.png - cluster_2b_floorplan.png

Cluster_3_elevation.png- cluster_3_elevation.png

Cluster_4_output.png - cluster_4_output.png

Cluster_5_review.png - cluster_5_review.png

═══

AGENT CLUSTERS — ALL ROUTED THROUGH THE AGENT MANAGER

═══

RULE: No agent calls another agent directly. Every input/output
handoff below — within a cluster AND across clusters — passes

through the Agent Manager, which sequences it, logs it, and enforces retry/loop limits.

CLUSTER 1 — SITE & CONSTRAINTS

Components:

- Location Zoning Agent (Kavya, Prem)
- Constraint Agent (Harshit, Dhwani)
- Entity Constraint Engine (Harshit, Anshul)

Flow:
Location Zoning Agent --[jurisdiction setbacks/coverage data]--> Constraint Agent
Constraint Agent --[per-entity rule requests]----------> Entity Constraint Engine
Entity Constraint Engine --[encoded rule set]----------> back to Constraint Agent

Where the output goes:
Final combined ruleset --> sent to Agent Manager --> Agent Manager releases it to Cluster 2 (Site Plan Generator + Floor Plan Generator) and Cluster 3 (Elevation Agent) whenever they request it.

Agent Manager's role here:
Holds the single source of truth handoff — makes sure every generator pulls the SAME version of the ruleset, so Location Zoning and Constraint Agent can never silently disagree.

CLUSTER 2 — GENERATE & VERIFY LOOP

Components:

- Site Plan Generator Agent (Devam)
- Floor Plan Generator Agent (Prasad, Devank)
- Json Extractor (Devank, Prasad)
- Verifier Agent (Devam, Harshit)
- Z3 Verifier (tool) (Swara, Ojas)

Flow:
Site Plan Generator --[footprint coordinates]--------> Floor Plan Generator

Floor Plan Generator --[room layout coordinates]------> Json Extractor
Json Extractor --[clean JSON file]----------> Verifier Agent
Verifier Agent --[calls]----------> Z3 Verifier (runs the proof)
Z3 Verifier --[pass/fail + violated constraints]--> back to Verifier Agent

Where the output goes:

- IF PASS --> Agent Manager forwards verified coordinates to Cluster 4 (Output & Export agents)
- IF FAIL --> Agent Manager routes Verifier's diagnostic feedback (e.g. "move house_top_y from 61 to 65") back to Site Plan Generator / Floor Plan Generator for a retry (loop capped — Agent Manager enforces max retry count)

Agent Manager's role here:
This is the busiest cluster. Agent Manager owns the retry loop — it decides when to send a failed design back, how many retries are allowed, and when to escalate to a human if it never converges.

CLUSTER 3 — ELEVATION & MULTI-FLOOR

Components:

- Topography (tool) (Devam, Marjit)
- Elevation Agent (Devam, Marjit)
- Multi-floor Generator Agent (Nitesh)

Flow:
Topography --[slope/contour/elevation data]----> Elevation Agent
Elevation Agent --[foundation height + strategy]---> Multi-floor Generator
Multi-floor Generator --[floor N layout]----------> back to Elevation Agent
(re-check if floor below needs adjustment)
(this repeats — settling loop — until both floors stabilize)

Where the output goes:
Final settled multi-floor structure --> Agent Manager --> forwarded to Cluster 4 (Side Views Agent + Roof Plan Agent) so upper-level views match the final floor heights.

Agent Manager's role here:
Enforces the settling-loop cap (no infinite back-and-forth

between floors) and is the one that flags to the user when a multi-floor design can't converge.

CLUSTER 4 — OUTPUT & EXPORT

Components:

- Side Views Generation Agent (Dhwani)
- Roof Plan Agent (Ojas)
- DXF Generator (Harshit, Devam)
- DXF file reader (Sidak, Saksham)
- CAD file reader (Siddh, Devam)
- PDF generation (report) (Anirudh)

Flow:
Verified floor/elevation data --> Side Views Agent --[elevation drawings]--> DXF Generator
Verified floor/elevation data --> Roof Plan Agent --[roof layout]--------> DXF Generator
DXF Generator --[final DXF file]----------> DXF file reader (for round-trip re-verification)
DXF Generator + sign-off data --[combined package]-----> PDF generation (report)
CAD file reader --[ingested reference design]--------> feeds back into Cluster 2 generators (for "similar to known layout" requests)

Where the output goes:
Final PDF + DXF package --> Agent Manager --> released to Cluster 5 (Reviewer City Agent + UI) for human sign-off.

Agent Manager's role here:
Makes sure Side Views, Roof Plan, and DXF Generator are all pulling from the SAME final verified coordinate set — so the three outputs never visually contradict each other.

CLUSTER 5 — HUMAN REVIEW CHAIN

Components:

- Planner/Customer Support Agent (Swara, Prem)
- Livability Agent (Aadi, Shreyas)

- Reviewer (City) Agent (Siddh, Rohan)
- UI (Triyansh, Bhavya)
- Notification Service (assigned separately, backlog)

Flow:
Planner Agent --[structured user brief]----------> Agent Manager
(kicks off Cluster 1 + Cluster 2)
Livability Agent --[daylight/airflow scores]---------> UI (shown alongside legal compliance)
Reviewer (City) Agent --[element-by-element sign-off]----> PDF generation (Cluster 4)
UI --[status updates]------------------> Notification Service
--[notifies]----------------------> Client / Architect / City Reviewer personas

Where the output goes:
Sign-off record --> Agent Manager --> triggers final report
release to the client through the UI.

Agent Manager's role here:
Keeps the three personas (client, architect, city reviewer)
in sync on the SAME pipeline state, since they're viewing the
same design at different stages of approval.

═══
═══
SUMMARY — CROSS-CLUSTER FLOW (ALL VIA AGENT MANAGER)

═══
═══
Cluster 5 (Planner)
--> Agent Manager
--> Cluster 1 (rules loaded)
--> Agent Manager
--> Cluster 2 (generate + verify loop, repeats until pass)
--> Agent Manager
--> Cluster 3 (elevation + multi-floor settling)
--> Agent Manager
--> Cluster 4 (exports: views, roof, DXF, PDF)
--> Agent Manager
--> Cluster 5 (Reviewer sign-off + UI + notifications back to client)

═══
═══

3) PIPELINE and I/O

Pipeline I/O reference — exact input/output format per agent

This is the companion reference to the pipeline flowchart. The flowchart shows *order*; this shows *exact data shape* — what every agent receives and what it must hand off.

All inter-agent messages use **JSON-RPC**, validated by **Pydantic** at the agent boundary. If an agent's output fails Pydantic validation, that agent's own LLM must reformat it before the output ever reaches the Agent Manager — invalid JSON should never leave an agent's own scope.

Tool calls (e.g. Site Plan Generator calling the Z3 Verifier tool directly) are **not** Agent Manager traffic — those stay inside the calling agent's own code and can use whatever format that agent's code needs internally.

---

1. Planner / Customer Support Agent — Prem (primary), Swara

Input: raw free-text user prompt (string)

Output (to Agent Manager):
```json
{
  "role": "planner",
  "status": "brief_ready" | "needs_clarification",
  "brief": {
    "square_footage": 2400,
    "bhk_count": 3,
    "budget_range": [400000, 550000],
    "location": {
      "city": "Redmond",
      "state": "WA",
      "zip": "98052"
    },
    "template_id": "3bhk_standard" | null
  },
  "clarification_question": "string, only present if status = needs_clarification"
}
```

Rule: never fills `location` with a guess. If a city name is ambiguous (e.g. "Redmond" — WA or CA), `status` must be `needs_clarification` until ZIP-level resolution is reached.

---

2. Agent Manager — Swara (primary), Prem

Input: every message from every other agent, in JSON-RPC envelope:
```json
{
  "jsonrpc": "2.0",
  "method": "agent.report_status" | "agent.request_input" | "agent.output_ready",
  "params": { "agent_id": "string", "payload": {} },
  "id": "request_id"
}
```

Output: routes the inner `payload` to the next agent in sequence, after Pydantic schema validation. Also emits a simplified status object to the Planner/Customer Support Agent on every state change:
```json
{
  "pipeline_stage": "verifying_footprint",
  "user_facing_message": "Checking your design against local building rules...",
  "percent_complete": 35
}
```

Rule: never forwards raw internal agent names, stack traces, or tool names to the user-facing message field.

---

3. Location Zoning Agent — Kavya (primary), Prem

Input:
Street address (similar to the input for location toolkit) eg:- one microsoft way, redmond, usa
Output:
```json
{
  "status": "ok",
  "jurisdiction": "Redmond, WA",
  "zone_code": "NR",
  "zone_type": "residential",
  "zone_description": "Neighborhood Residential",
  "overlay_districts": [],

  "setbacks_ft": {
```

```
  "front": 10.0,
  "rear": 5.0,
  "side": 3.0
},
"side_street_setback_ft": 10.0,
"alley_setback_ft": 2.0,
"garage_setback_ft": 3.0,
"waterbody_setback_ft": null,
"building_separation_ft": 5.0,

"max_lot_coverage_pct": 50.0,
"max_impervious_surface_pct": 70.0,
"min_open_space_pct": 20.0,
"min_landscaping_pct": null,
"max_hardscape_pct": 25.0,
"min_green_factor_score": null,

"max_height_ft": 38.0,
"max_height_ft_nonresidential": null,
"max_height_ft_with_incentives": null,
"min_height_ft": null,
"max_stories": null,
"ground_floor_min_ceiling_ft": null,
"upper_level_stepback_ft": null,

"max_far": null,
"max_far_with_incentives": null,
"density_limit_du_per_acre": null,
"min_lot_size_sqft": null,
"max_units": 6.0,
"max_adus": 2.0,
"min_commercial_floor_area_sqft": null,

"min_residential_parking_spaces": null,
"min_commercial_parking_spaces": null,
"bicycle_parking_required": false,

"facade_transparency_min_pct": null,
"blank_facade_max_width_ft": null,
"modulation_required": false,

"zero_lot_line_allowed": true,
"tree_buffer_ft": 5.0,
"min_tree_retention_pct": 35.0,
```

```
  "tree_points_per_500_sqft": null,
  "tree_protection_zones": [],

  "source_url": "https://redmond.municipal.codes/RZC/21.12.500",

  "parcel_id": "1234567890",
  "parcel_area_sqft": 8500.25,
  "building_count": 1,
  "building_area_sqft": 1450.0,
  "building_file": "outputs/15670_ne_85th_st_buildings.json",
  "geometry_file": "outputs/15670_ne_85th_st_geometry.json"
}


```

Rule: if no constraint data exists for the resolved ZIP, return `"jurisdiction": null` with an explicit `"error": "no_constraint_data"` field — never silently substitute a default ruleset.

---

4. Constraint Agent + Entity Constraint Engine — Harshit (primary), Dhwani / Anshul

Input: jurisdiction data from Location Zoning Agent (above)

Output:
```json
{
  "entity_rules": {
    "bedroom": { "min_area_sqft": 70, "must_connect_to": ["hallway"] },
    "bathroom": { "min_area_sqft": 35, "not_adjacent_to": ["bathroom", "kitchen"] },
    "kitchen": { "min_area_sqft": 80 },
    "living_room": { "min_area_relative": "> any bedroom" }
  },
  "jurisdiction_overrides": { "...same shape as above, only differing fields" }
}
```

Rule: this is the single source of truth — both the Floor Plan Generator and the Verifier Agent must query this exact same object, never a locally cached copy.

---

5. Guardrail Tools — Swara

Input: the raw planner brief (before generation starts)

Output:
```json
{ "allowed": true | false, "reason": "string, only if allowed = false" }
```

Rule: runs before the Constraint Agent's geometric checks, not after. A blocked request never reaches the generator agents.

---

6. Topography (tool, conditional) — Devam (primary), Marjit

Input: lot coordinates from Location Toolkit

Output:
```json
{
  "elevation_data_available": true,
  "max_elevation_diff_ft": 14,
  "contours": [ { "elevation_ft": 120, "polygon": [[x,y], ...] } ]
}
```

Rule: if `elevation_data_available` is false, every downstream agent must treat the lot as flat explicitly — not by omission.

---

7. Site Plan Generator Agent — Devam (primary), Ayush

Input: combined ruleset (setbacks, coverage, tree zones) + topography data if present

Output:
```json
{
  "footprint_polygon": [[x,y], [x,y], [x,y], [x,y]],
  "lot_coverage_pct": 33.2,
  "buildable_area_sqft": 2640
}
```

Tool call (internal, not Agent Manager traffic): calls Z3 Verifier directly to check this output before returning it.

---

8. Verifier Agent + Z3 Verifier (tool) — Devam/Harshit (Verifier Agent); Ayush (Z3 Verifier)

Input: any generator's coordinate output (footprint or room layout) + the constraint ruleset

Output, on pass:
```json
{ "result": "pass", "verified_coordinates": { "...same shape as input" } }
```

Output, on fail:
```json
{
  "result": "fail",
  "violations": [
    { "constraint": "setback_rear", "current_value": 12, "required_value": 15,
      "fix_suggestion": "move house_top_y from 61 to 65" }
  ]
}
```

Rule: `fix_suggestion` is diagnostic only — the Verifier never returns corrected coordinates directly, or the Generator↔Verifier loop collapses into one step.

---

9. Floor Plan Generator Agent — Prasad (primary), Devank

Input: `verified_coordinates` (the footprint) + entity rules from Constraint Agent

Output:
```json
{
  "rooms": [
    { "id": "bed_1", "type": "bedroom", "polygon": [[x,y],...], "area_sqft": 140 },
    { "id": "bath_1", "type": "bathroom", "polygon": [[x,y],...], "area_sqft": 45 }
  ],
  "corridor_area_sqft": 180,
  "total_area_sqft": 2400
}
```

```

**Tool call (internal):** hands this to Json Extractor before it goes to the Verifier Agent.

---

## 10. Json Extractor (tool) — Devank (primary), Prasad

**Input:** raw generator output (any shape, possibly malformed)

**Output (clean JSON, passed to Verifier):**
```json
{ "rooms": [ "...same shape as Floor Plan Generator output above, validated and re-typed" ] }
```

Output (on malformed input):
```json
{ "error": "malformed_generator_output", "raw_input_excerpt": "string" }
```

Open question (unresolved as of latest meeting): scope vs. Pydantic-level validation needs clarifying with Chandra — may overlap.

---

11. Multi-floor Generator Agent (conditional) — Nitesh

Input: verified floor 1 layout + Elevation Agent's foundation height data

Output:
```json
{
  "floors": [
    { "floor_number": 1, "rooms": ["...same room shape as Floor Plan Generator"] },
    { "floor_number": 2, "rooms": ["..."], "wall_alignment_check": "pass" | "fail" }
  ],
  "converged": true | false
}
```

Rule: if `converged` is false after the retry cap, the Agent Manager must surface this to the user — never silently return a mismatched-floor design.

```

12. Elevation Agent — Ayush (primary), Devam

Input: topography data + footprint

Output:
```json
{
  "foundation_strategy": "dig_down" | "elevate_fill",
  "foundation_height_ft": 3.5,
  "reasoning": "string"
}
```

---

13. Livability Agent (parallel, non-blocking) — Aadi (primary), Shreyas

Input: verified room layout

Output:
```json
{
  "room_scores": [
    { "room_id": "bed_1", "daylight_score": 0.82, "airflow_score": 0.71 }
  ]
}
```

Rule: never blocks the pipeline — this is informational, attached to the final report, not a pass/fail gate.

---

14. Side Views Agent — Dhwani

Input: verified floor + elevation data

Output: DXF-layer-ready coordinate set for front/rear/side facades (same coordinate format as floor plan, tagged `"view_type": "elevation"`)

15. Roof Plan Agent — Ojas

Input: top-floor footprint + elevation data

Output: roof polygon set tagged `"view_type": "roof"`, must exactly match top-floor boundary

16. DXF Generator (tool) — Harshit (primary), Devam

Input: any verified coordinate set tagged with a `view_type` (floor / elevation / roof)

Output: `.dxf` binary file + manifest:
```json
{ "file_path": "string", "layers": ["floor_1", "elevation_front", "roof"] }
```

Rule: only ever called on already-verified (post Z3 pass) coordinates.

17. DXF file reader (tool) — Sidak (primary), Saksham

Input: `.dxf` file path

Output: same coordinate JSON shape that produced it (round-trip, zero data loss)

18. CAD file reader (tool) — Siddh (primary), Devam

Input: external CAD file (for "similar to known layout" requests)

Output: ingested polygon/room data in the standard coordinate format, fed back into Site Plan / Floor Plan Generators as a reference seed

19. PDF generation (report) — Ayush (primary), Anirudh

Input: all verified outputs (floor plan, elevations, roof, Reviewer's sign-off record)

Output: `.pdf` file meeting: min 300 DPI, every door/window labeled, N/S/E/W directional labels present, formatted per CommonFloor/MagicBricks-style detailing conventions

---

20. Reviewer (City) Agent — Siddh (primary), Rohan

Input: complete output package (floor plan + elevations + roof + PDF draft)

Output:
```json
{
  "element_signoffs": [
```

```
    { "element_id": "wall_north_1", "approved": true },
    { "element_id": "setback_rear", "approved": false, "reason": "string" }
  ],
  "overall_status": "approved" | "rejected"
}
```

Rule: any environmental/zoning violation results in automatic `"approved": false` — no override path.

---

21. UI + Notification Service

Input: Agent Manager's simplified status stream + Reviewer's sign-off result

Output: persona-scoped notifications — client, architect/HITL, and city reviewer each receive only the events relevant to their role; never cross-leaked.

Always-on background services (queried on demand, not pipeline steps)

Service	Input	Output
Database (Supabase + Qdrant)	read/write keyed by user_id + session_id	persisted records; vector search returns ranked matches with similarity score
Optimizations/Keys/Auth	agent_id + request	valid API key + usage counter increment
Deployment Engine	n/a (infra layer)	health/status endpoint consumed by Agent Manager
End-End Validation	full scenario definition	pass/fail report per scenario, with which agent/step failed

---

Reference this alongside the pipeline flowchart image. If any agent's actual output ever needs a field not listed here, update this doc and the Sprint plan 186 tab's checklist column together so they don't drift apart.

22)Door agent-
INPUT-
```
{
  "verified_floor_plan": {
    "rooms": [
      {
```

```
      "id": "bed_1",
      "type": "bedroom",
      "polygon": [[x, y], [x, y], [x, y], [x, y]],
      "area_sqft": 140
    }
   ],
   "corridor_area_sqft": 180,
   "total_area_sqft": 2400
  },
  "connectivity_rules": {
   "bedroom":     { "must_connect_to": ["hallway"] },
   "bathroom":    { "must_not_open_to": ["kitchen"] },
   "kitchen":     {},
   "living_room": { "must_have_exterior_entry": true },
   "hallway":     {}
  },
  "occupied_wall_intervals": [
   {
    "room_id": "bed_1",
    "wall_index": 0,
    "intervals": [
     { "start_offset_ft": 1.5, "end_offset_ft": 4.5, "occupied_by": "window" }
    ]
   }
  ]
}
```

OUTPUT-

```
{
  "role": "door_agent",
  "status": "ok" | "error",
  "doors": [
   {
    "door_id": "door_bed1_hall",
    "type": "interior" | "exterior",
    "connects": ["bed_1", "hallway"],
    "wall": {
     "room_a_id": "bed_1",
     "room_b_id": "hallway",
     "wall_start": [x1, y1],
     "wall_end":   [x2, y2]
    },
    "placement": {
```

```
      "offset_from_wall_start_ft": 3.0,
      "width_ft": 3.0,
      "swing_direction": "inward" | "outward"
    },
    "is_primary_entry": false
  }
 ],
 "connectivity_graph": {
  "nodes": ["bed_1", "bath_1", "hallway", "living_room", "kitchen"],
  "edges": [
   { "door_id": "door_bed1_hall", "room_a": "bed_1", "room_b": "hallway" }
  ],
  "all_rooms_reachable": true
 },
 "error": "string, only present if status = error"
}
```